RAMDEOBABA UNIVERSITY

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

B. TECH PROJECT REPORT

Idea Lab

ON

CLINICAL UNDERSTANDING AND RISK EVALUATION

(CURE)

Submitted by

Student Name 1 (Roll No. _____)

Student Name 2 (Roll No. _____)

Student Name 3 (Roll No. _____)

Student Name 4 (Roll No. _____)

Under the Guidance of

Guide Name: _____

Department of Computer Science and Engineering

Ramdeobaba University, Nagpur

Academic Year: 2026 - 27

CERTIFICATE

This is to certify that the project report entitled **"Clinical Urgency and Risk Evaluation (CURE)"** is a bona fide work carried out by the students named on the title page, submitted to Ramdeobaba University, Nagpur, in partial fulfilment of the requirements for the Bachelor of Technology degree.

The work described here is an academic software prototype. Implementation status and reported checks are based on the project repository and handoff records. This report does not claim clinical validation or readiness for real-patient use.

Project Guide: _____ Signature: _____

Project In-Charge: _____ Signature: _____

Date: _____

Place: Nagpur

STUDENT DECLARATION

We declare that this report describes the CURE project work completed by our team for the B.Tech programme. The implementation status, test results, and limitations are stated from project records.

External software, documentation, and public data sources are acknowledged in the references. The NHAMCS model results are research-only. No clinical performance or patient benefit is claimed.

Student Name 1: _____ Roll No.: _____

Student Name 2: _____ Roll No.: _____

Student Name 3: _____ Roll No.: _____

Student Name 4: _____ Roll No.: _____

Date: _____

APPROVAL SHEET

This report entitled **"Clinical Urgency and Risk Evaluation (CURE)"** by the students listed on the title page is submitted for the Bachelor of Technology degree.

Approved by:

Project Guide: _____ Signature: _____

Associate Head: _____ Signature: _____

Dean, School of Computer Science and Engineering: _____

Date: _____

Place: Nagpur

ACKNOWLEDGEMENT

We thank Ramdeobaba University and the School of Computer Science and Engineering for the academic setting in which this project was developed.

We are grateful to our project guide and faculty members for their guidance and feedback. We also acknowledge:

- the maintainers of the open-source tools used in the prototype; and
- the National Center for Health Statistics (NCHS) for its public-use NHAMCS data and documentation.

The project team takes responsibility for accurately describing the system scope, prototype checks, and safety limitations in this report.

ABSTRACT

CURE (Clinical Understanding and Risk Evaluation) is an academic web prototype for organizing a preliminary patient intake workflow. It combines a React interface, a FastAPI service, and PostgreSQL storage, and provides four role workspaces: patient intake, triage routing, assigned-doctor review, and administrator account management with aggregate analytics.

Key capabilities of the prototype:

- Patients maintain a profile and structured history, and submit an adaptive symptom questionnaire.
- PDF or image reports can be uploaded for local text extraction and patient verification.
- Screening records preserve a snapshot of responses and relevant history.
- Independent rule-based emergency flags take precedence over ordinary routing.
- Triage staff route a screening to a doctor, who can record a review.

Evidence recorded in the handoff:

- An 18-test passing backend suite and a successful frontend build.
- Healthy database and API checks; synthetic upload samples accepted by the local extraction path.
- A separate research pipeline compared logistic-regression and random-forest baselines on CDC NHAMCS 2022 data. Rare-class performance is weak, and no model is deployed in CURE.

CURE is not a diagnostic tool or emergency-care service. Emergency rules require qualified clinician review, and the experimental model is not clinically validated. The prototype is for academic demonstration with fictional data only.

Keywords: clinical intake, role-based access, screening workflow, FastAPI, React, PostgreSQL, safety flags

TABLE OF CONTENTS

Preliminary pages: Certificate; Student Declaration; Approval Sheet; Acknowledgement; Abstract; Lists of Figures, Tables and Abbreviations.

Chapter 1: Introduction 6
Chapter 2: Literature Review and Research Gap 7
Chapter 3: Proposed System and Methodology 8
Chapter 4: System Design and Implementation 10
Chapter 5: Results and Discussion 12
Chapter 6: Conclusion and Future Scope 14
References 15
Annexure A: Publications and Intellectual Property 16
Annexure B: Student Contribution Statement 16
Annexure C: Project Outputs 17

LISTS OF FIGURES, TABLES AND ABBREVIATIONS

List of Figures

- Figure 3.1 CURE high-level workflow
- Figure 3.2 Screening lifecycle
- Figure 4.1 Service architecture and data flow
- Figure 5.1 Research baseline test-set comparison

List of Tables

- Table 2.1 Related system concerns and CURE response
- Table 4.1 Main software components
- Table 4.2 Principal data entities
- Table 5.1 Verification summary
- Table 5.2 Experimental ML baseline results

Abbreviations

Abbr.	Full form	Abbr.	Full form
API	Application Programming Interface	JWT	JSON Web Token
CRUD	Create, Read, Update, Delete	ML	Machine Learning
ED	Emergency Department	OCR	Optical Character Recognition
HTTP	Hypertext Transfer Protocol	RBAC	Role-Based Access Control
NHAMCS	National Hospital Ambulatory Medical Care Survey	UI	User Interface

CHAPTER 1 - INTRODUCTION

1.1 Background

Healthcare intake information may arrive through different channels and can be difficult to organize consistently. A software prototype can demonstrate structured intake, access boundaries, review routing, and traceable handling of submitted information. CURE was developed as an academic workflow prototype around these needs.

The system is scoped to intake organization and workflow demonstration. It does not diagnose conditions, prescribe care, or replace professional judgment. The interface instructs users with emergency symptoms to seek emergency help rather than waiting for CURE or a clinician response.

The implementation connects the following elements into one accountable chain:

- patient-submitted responses;
- report extraction that remains unverified until confirmed;
- a screening-time snapshot;
- rule-based emergency flags;
- staff routing; and
- doctor review.

The prototype treats a screening as a sequence of accountable steps rather than a single score. A patient supplies information, the service validates and records it, and staff members act within their assigned responsibilities. This makes it possible to show where information originated, what was available at submission time, and which role handled the next step.

The project is also an exercise in communicating boundaries. A polished interface must not imply that a rule or model has diagnosed a person. CURE therefore separates workflow status from clinical truth and makes escalation language visible when a configured urgent pattern is present.

1.2 Problem Statement

To demonstrate how patient intake, report handling, screening persistence, triage routing, and assigned clinician review can be connected under role-based access controls while communicating uncertainty and urgent escalation.

1.3 Project Objectives

- Implement patient, triage, doctor, and administrator workflows.
- Persist profiles, reports, screenings, snapshots, routing, reviews, and audit events.
- Enforce server-side authorization and patient-specific access boundaries.
- Provide local PDF/image extraction with patient verification.
- Keep emergency rules independent from experimental ML.
- Evaluate software behavior and a separate research-only ML baseline.

1.4 Scope and Limitations of Scope

Within scope:

- Local Docker Compose operation.
- Synthetic demo files.
- Regression tests.
- Exploratory model evaluation.

Outside scope:

- Clinical rule approval.
- Predictive model deployment.

- Production compliance operations.
- Real-world clinical use (use fictional data only).

1.5 Intended Demonstration

The demonstration begins with a patient account and a fictional profile. The sequence is:

- The user adds history, uploads a synthetic report, and checks any extracted fields.
- The user submits the questionnaire.
- Staff follow the saved screening through a triage decision and an assigned-doctor review.
- An administrator demonstrates account controls and aggregate operational information.

Success at this scope means the records can be followed through the prototype and restricted by role, not that the system can determine the correct care pathway. The team should present the software as an educational workflow demonstration, describe test evidence accurately, and call out unfinished clinical and production work.

CHAPTER 2 - LITERATURE REVIEW AND RESEARCH GAP

This chapter summarizes the engineering concepts CURE builds on and identifies the gap the project addresses. CURE applies established software patterns rather than proposing a new clinical method.

2.1 Literature Review

Layered web architecture. A browser client communicates with an application API. The API authenticates requests, validates input, applies role checks, and persists state in a relational database. Separating interface, service, and storage supports independent validation.

Capture versus interpretation. Clinical workflow software must distinguish data capture from clinical interpretation. Extracted text is not automatically a verified clinical fact, and a screening is not a diagnosis. CURE separates raw extraction from patient-verified fields and displays a non-diagnostic disclaimer.

Dataset labels as proxies. A dataset label is only a proxy for an intended outcome unless its meaning and population fit the task. NHAMCS is a survey of ambulatory-care utilization, not a prospective CURE cohort. Historical provider triage categories cannot establish safety or effectiveness of a patient-facing urgency system.

Relational persistence. Relational persistence suits workflows in which records have explicit ownership and relationships. Patient, screening, report, assignment, review, and audit records can be linked with identifiers and constraints. A server-side API can then check the authenticated role and record relationship before returning protected content, which reduces reliance on client-side visibility as a security boundary.

Machine-learning evaluation. Evaluation must reflect the intended task and the distribution of relevant cases. A single aggregate score can obscure errors in uncommon but consequential categories. Reporting per-class results and a held-out test score makes these weaknesses more visible, while still not resolving the mismatch between a retrospective survey label and a prospective clinical decision.

2.2 Existing Approaches and Research Gap

Existing digital intake systems commonly combine forms, account management, storage, and staff review. Recurring engineering concerns include missing answers, inconsistent uploads, authorization mistakes, unclear record ownership, and uncertainty about automated outputs.

Table 2.1 - Concern and CURE response

Concern	CURE response
Unstructured intake	Adaptive questionnaire with explicit missingness.
Unverified extraction	Raw extracted text is separated from patient-verified fields.

Concern	CURE response
Fragmented review	Triage queue, routing, doctor review, and audit events.
Unsafe automation claims	Independent warning rules, visible caveat, no deployed ML model.

The engineering gap addressed is an academic end-to-end demonstration connecting these software components with role-based controls and a documented boundary between a workflow prototype and clinically validated care. This is an integration objective, not a claim of clinical novelty.

Interpreting the table:

- The responses are engineering safeguards, not proof that every operational risk is solved.
- Explicit unknown answers improve data interpretation, but staff still decide whether missing information should trigger follow-up.
- A separated verification state makes provenance clearer, but it does not establish the accuracy of patient-confirmed content.

Accordingly, the project contribution is an integrated, inspectable prototype with documented limitations. The report avoids claims of novelty in triage science, clinical accuracy, or patient outcomes. Its evaluation asks whether the intended software path is represented and whether key authorization and persistence behaviors are covered by regression checks.

CHAPTER 3 - PROPOSED SYSTEM AND METHODOLOGY

3.1 Proposed System and Requirements

CURE has four roles: patient, triage staff, doctor, and administrator. Patient registration creates patient accounts; privileged accounts are managed through administrator functions. Authentication uses password hashing and signed tokens, and backend authorization checks protect operations rather than relying on the interface alone.

Functional requirements:

- Profile and history management.
- Report upload and verification.
- Questionnaire completion and screening history.
- Urgent rule flags.
- Triage routing and assigned-doctor review.
- Account management and aggregate analytics.
- Audit logging.

Quality (non-functional) requirements:

- Input validation and understandable errors.
- Bounded file handling for uploads.
- Maintainable route and service organization.
- Repeatable local setup for demonstration.

Figure 3.1 - CURE high-level workflow

Patient intake → Screening and safety checks → Triage queue → Doctor assignment and review → Aggregate administration

Emergency warnings are surfaced immediately and do not wait for ordinary routing.

A typical request passes through a fixed series of checks:

- the API verifies the token;
- the payload is validated;

- the role and resource relationship are confirmed; and
- the relevant workflow operation is invoked.

The interface provides role-specific navigation, but the backend remains responsible for deciding whether a request is permitted. A hidden button alone cannot protect an API endpoint. The Compose configuration is intended for a development demonstration and should not be described as a production deployment, because monitoring, operational security, backup strategy, and formal privacy processes require additional work.

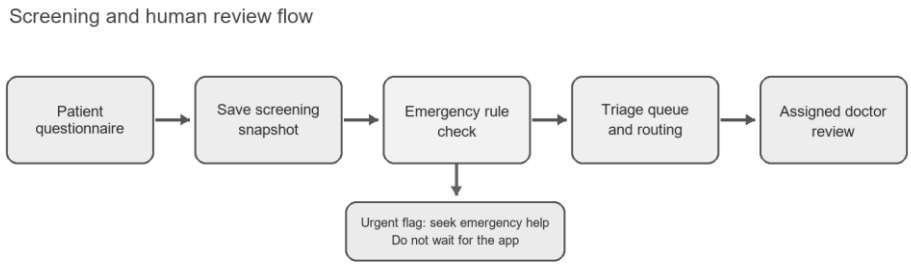
3.2 Methodology and Screening Lifecycle

The adaptive questionnaire covers general, respiratory, digestive, neurological, and cardiovascular symptoms. It captures severity, duration, progression, and associated symptoms where applicable. Optional free text and unanswered items are represented without treating unknown values as negative.

On submission, the service stores responses and an immutable snapshot of relevant patient history. The lifecycle then proceeds as follows:

- Safety rules are evaluated independently of the research model.
- Emergency-pattern flags take priority.
- Only screenings without an urgent flag proceed through routine triage.
- Triage routing records an assignment.
- An assigned doctor can record a review, with an audit trail.

Figure 3.2 - Screening lifecycle



Draft intake → validated submission → persisted snapshot → emergency precedence check → routine triage → doctor review

Persisting a snapshot supports later interpretation of the screening in the context in which it was submitted. If a patient edits their profile afterward, the historical submission remains associated with its original captured context. This is a data-integrity feature; it does not guarantee that the submitted history was complete or clinically correct.

The urgent branch is deliberately independent of the experimental model. A configured rule can display an emergency warning, while routine routing applies only when no such warning is raised. The user-facing message tells people not to wait for CURE or a clinician response. The rule definitions and wording still need review by qualified clinicians before any real-patient use.

3.3 Dataset and ML Evaluation Method

The isolated research pipeline uses the 2022 CDC National Hospital Ambulatory Medical Care Survey Emergency Department public-use file. The handoff records inspection of variables, class distribution, missingness, label semantics, and limitations. The target is historical provider-recorded triage level 1-5, a proxy label rather than a patient outcome or CURE urgency.

Evaluation design:

- Logistic regression and random forest are compared.
- Preprocessing is leakage-conscious.
- Evaluation uses a stratified holdout with per-class reporting.

- Validation selects a candidate; the held-out test set is reported separately.
- Artifacts are local research outputs and are not loaded by the application.

NHAMCS was selected as an available public source for exploratory engineering work, not because it is a direct match for CURE deployment. Its survey design, encounter context, recorded fields, and provider-assigned triage category shape what the models can learn. Preprocessing and class accounting are therefore documented alongside model scores so that results are not read as a general-purpose urgency estimate.

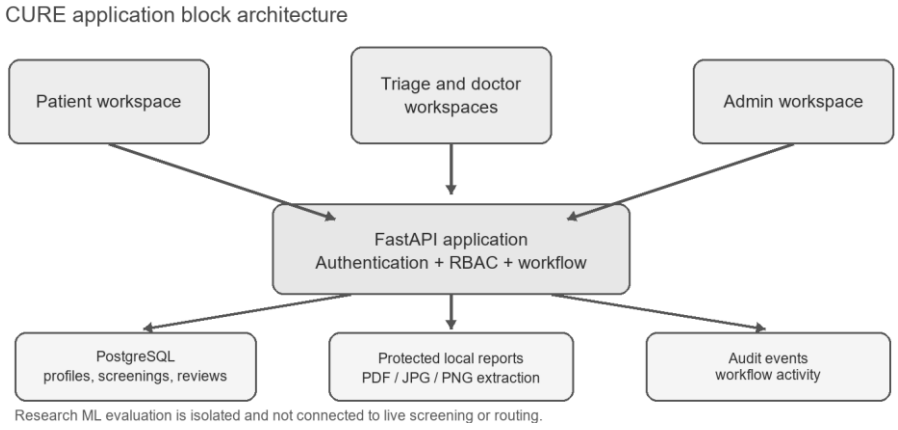
The holdout split is used to report a final comparison after candidate selection on validation data, which reduces the chance of choosing a model solely on its test performance. It cannot remove dataset shift, label ambiguity, selection bias, or the need for independent evaluation on a population and task aligned with an intended use. Rare-class performance is poor, and the data context does not validate CURE use. Further research requires suitable data, external evaluation, expert review, and assessment of calibration, subgroup performance, and failure modes.

CHAPTER 4 - SYSTEM DESIGN AND IMPLEMENTATION

4.1 System Architecture

The prototype is a modular monolith deployed with Docker Compose. The React/TypeScript frontend serves role-specific pages, the FastAPI backend exposes versioned API routes and service logic, PostgreSQL stores structured data, and protected local storage holds uploaded originals.

Figure 4.1 - Service architecture and data flow



Browser UI ↔ FastAPI API ↔ PostgreSQL; FastAPI also accesses protected local report storage. Authentication and role authorization guard API operations; audit events record important actions; no predictive inference is connected.

Table 4.1 - Main software components

Component	Technology / role
User interface	React and TypeScript
Backend API	FastAPI and Python
Database	PostgreSQL
Schema migrations	Alembic
Deployment	Docker Compose
Report text extraction	Local PDF/image text extraction
Research pipeline	Isolated Python / scikit-learn

Health checks and service dependencies support local startup. Development secrets use an ignored .env file. This is local prototype infrastructure, not hardened production deployment.

A request originating in a role workspace is sent to the API over the local development network. The backend coordinates validation and workflow logic, then reads or writes the corresponding database records. Protected file handling for reports is mediated by the backend rather than exposing the storage directory as a public static folder, and audit records capture selected sensitive operations for later review.

This architecture keeps the research experiment outside the live request path. Training and evaluation run as a separate command-line workflow, and generated artifacts remain local. The application does not load model files or use their predictions to order or route screenings, which is intentional given the recorded weaknesses and lack of clinical validation.

4.2 Data Model and Storage

Principal entities include accounts and roles, patient profiles and histories, report metadata, extracted and verified fields, screening records and snapshots, doctor-patient assignments, triage routing, clinician reviews, model metadata, and audit events.

Table 4.2 - Principal data entities

Entity	Purpose
Account	Stores identity and role state.
Patient profile	Stores patient information.
Report	Stores protected metadata and verification state.
Screening	Stores responses, flags, and history snapshot.
Assignment	Links a doctor and a patient.
Review	Stores the doctor workflow response.
Audit event	Records sensitive actions.

Design properties:

- Constraints and server-side ownership checks support isolation.
- Screening snapshots preserve submission-time context if profile data later changes.
- Uploads are bounded and validated.
- Account role and active state set broad permissions; patient ownership and doctor assignment narrow access to individual records.
- The report record separates metadata and verification state from the protected original file.

Known limitation: durable cleanup of all dependent files after deletion is not yet implemented. Removing a database record does not by itself ensure that every dependent file has been cleaned up. A production system would need a defined retention schedule, a reliable cleanup process, monitoring, backups, recovery procedures, and a reviewed process for user requests.

4.3 API, Authentication and Authorization

Versioned backend routes cover authentication, patient information, questionnaires, screenings, reports, triage, doctor workflow, and administration. Requests are validated before persistence. Passwords are stored with a password-hashing function rather than plaintext, and signed JWTs carry authenticated identity.

RBAC is enforced at API and service boundaries:

Role	Permitted access
Patient	Own profile, reports, and screenings.
Doctor	Assigned patients; can record reviews.
Triage staff	Process queue items and route screenings.

Role	Permitted access
Administrator	Manage accounts and view aggregate analytics.

Role checks are paired with resource checks: a patient identity must match the owner of a requested record, a doctor must have an assignment before accessing the corresponding workflow, triage actions are limited to triage operations, and administrative operations require the administrator role. Sensitive report access is restricted and audited.

The integration suite covers the following areas:

- authorization boundaries and assignment isolation;
- workflow permissions and audit behavior;
- emergency precedence and snapshots;
- report failures and account management.

Authentication and authorization are separate controls: a valid login establishes an identity, while each protected operation still needs permission checks. Passing prototype tests do not certify production security or regulatory compliance. A production deployment would also need a reviewed key-rotation plan, transport security, secret management, abuse monitoring, and independent security assessment.

4.4 User Interface and Report Handling

Each role has a dedicated workspace:

- **Patient:** profile/history data, report upload and verification, questionnaire submission, and screening history.
- **Triage:** queue and routing controls.
- **Doctor:** assigned work and review controls.
- **Administrator:** account management and aggregate analytics.

PDF, JPG, and PNG examples are synthetic. Extracted text remains unverified until the patient confirms structured content. Upload errors and limitations are surfaced, and original files are protected in local storage with access depending on role and assignment. Users should inspect extracted values before confirming them, since layouts, scans, abbreviations, and image quality can affect extraction.

The interface states that CURE is not a diagnostic system or emergency service. An urgent warning instructs the user to seek emergency help immediately and not wait for the application. Rules require qualified review before real-world use.

Recommended demonstration order:

- Patient intake.
- Triage queue.
- Assigned-doctor view.
- Administrator account or aggregate functions.

Using separate demo accounts clarifies which actions are available to each role. A successful navigation demo is not a replacement for formal usability testing or a complete authenticated end-to-end verification.

CHAPTER 5 - RESULTS AND DISCUSSION

5.1 Experimental Setup and Verification

The local environment uses Docker Compose with React, FastAPI, and PostgreSQL. The handoff dated 23 October 2026 records the results summarized below. These checks establish prototype behavior in the recorded environment, not full authenticated browser verification of every role.

Table 5.1 - Verification summary (values are handoff records)

Check	Recorded result
Backend integration suite	18 passed (one Starlette deprecation warning)
Frontend production build	Passed
Database	Healthy
API health	"ok"
Frontend	HTTP 200
Sample extraction	Non-empty text for three synthetic files (PDF, PNG, JPG)

Verification evidence is intentionally separated by type, and each result answers a limited question about the prototype:

- Automated backend tests exercise API-level behavior and authorization rules.
- A frontend production build checks compilation and bundling.
- Service health and HTTP responses confirm the local containers started and served basic routes.
- Synthetic report checks exercise extraction on sample formats.

A fuller demonstration should use separate fictional accounts for patient, triage, doctor, and administrator roles, follow one screening through the journey, and confirm that restricted records are not visible to unrelated users. The handoff states that full authenticated role-to-role browser verification was not performed, so it remains a manual verification item.

5.2 ML Results

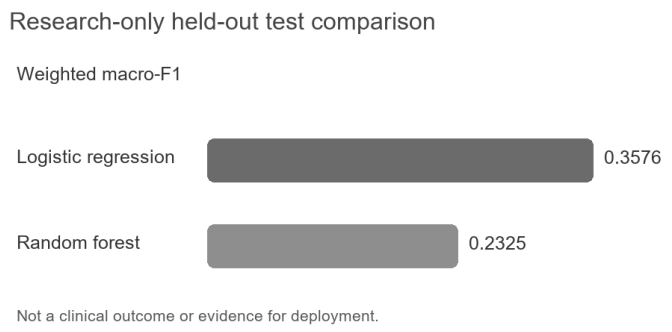
Table 5.2 - Experimental ML baseline results (weighted macro-F1)

Model	Validation	Held-out test
Logistic regression	0.3154	0.3576
Random forest	0.2259	0.2325

These results describe agreement with historical survey labels only. The selected baseline has inadequate rare-class recall, as the handoff reports:

- 4 of 21 immediate cases recalled.
- 5 of 46 nonurgent cases recalled.

Figure 5.1 - Research baseline test-set comparison



Test comparison: LR 0.3576; RF 0.2325. Low scores and weak rare-class recall support non-deployment.

Weighted macro-F1 is an exploratory summary across the historical triage classes and should be read together with the confusion matrix and class-level recall, since a seemingly moderate aggregate can coexist with missed uncommon classes. Aggregate metrics cannot support deployment, and the model remains disconnected from CURE.

The comparison provides a reasoned non-deployment decision. No threshold or output should be presented as an individual patient risk estimate. Better performance claims would require:

- an appropriate target;
- representative and governed data;
- clinician-defined evaluation criteria;
- external testing;
- calibration and subgroup analyses; and
- a safety review.

5.3 Discussion and Limitations

The implementation demonstrates integration of intake, structured data, report handling, screening snapshots, independent safety rules, role-based routing, review, and audit. Regression checks target important authorization and workflow failures, and synthetic files make upload behavior demonstrable without real health records.

Remaining limitations:

- Emergency rules require clinician review.
- NHAMCS proxy labels and rare-class performance do not justify deployment.
- The research model is not integrated.
- Full authenticated role-by-role browser verification remains.
- Durable file deletion, production privacy operations, deployment hardening, and independent compliance review remain outstanding.

The project is not clinically complete. Reported counts and metrics come from the handoff, and software checks do not imply safety, effectiveness, fairness, calibration, or suitability for patient-care decisions.

The most important distinction is between software verification and clinical validation. A passing authorization test can show that a defined request is rejected or allowed under test conditions. It cannot show that the emergency rule is medically complete, that the questionnaire captures all relevant context, or that a routing outcome improves care.

The prototype is best used for a controlled academic demonstration with fictional records. Before any broader use, the team would need expert review of urgent guidance, a privacy and retention plan, documented threat analysis, deployment hardening, accessible usability evaluation, and independent assessment of the data and intended clinical purpose.

CHAPTER 6 - CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

CURE demonstrates an academic intake and review workflow using React, FastAPI, and PostgreSQL. The prototype provides:

- patient, triage, assigned-doctor, and administrator workspaces;
- role-specific server-side controls;
- screening snapshots;
- protected local report upload and verification; and
- independent urgent warnings.

The project also includes regression coverage and a separate reproducible research pipeline. Recorded checks support basic prototype operation. NHAMCS results are weak for rare urgency classes and the model is not connected to the application. The outcome is a demonstrable software prototype, not a validated clinical product.

Objectives were achieved at prototype scope. The implementation brings them together in one demonstrable workflow, making software boundaries and role responsibilities visible and recording limitations alongside the implementation. Clinical rules, production operations, and model validity require qualified external review before any real-patient consideration.

The final deliverable should be described as a working academic prototype based on the verified local checks, with the scope stated clearly. It is not a completed clinical decision-support product. That distinction is central to presenting the work honestly and to preventing a demonstration interface from being mistaken for medical advice.

6.2 Future Scope

The items below are proposals, not completed features.

Safety and operations (first priority):

- Qualified clinician review of emergency rules and urgent guidance.
- Authenticated role-by-role end-to-end verification.
- Durable deletion and retention operations.
- Security, privacy, and compliance review for any intended deployment.

Research extensions:

- Examine better-matched datasets under approved access.
- Refine label definitions with clinicians.
- Evaluate calibration and subgroup behavior.
- Perform independent external validation; any model should remain isolated until evidence supports a bounded intended use and safe human oversight.

Engineering enhancements:

- Monitored deployment, backup, and recovery.
- Accessibility review, including keyboard accessibility.
- Observability and retention enforcement.
- Secure report lifecycle management.

The next practical step for a student demonstration is to complete a manual role walkthrough using fictional accounts and record any usability defects. Engineering work can then prioritize reliable file lifecycle cleanup, deployment configuration, monitoring, and clear retention behavior, each planned and verified individually rather than assumed from local startup success.

Clinical work must be governed separately from software development. Qualified clinicians should review the emergency rules and intended workflow, and any future model study should define a clinically meaningful target and appropriate oversight before collecting or using data. If evidence remains inadequate, keeping model inference out of the product is the correct outcome.

REFERENCES

[1] NCHS, "2022 National Hospital Ambulatory Medical Care Survey Emergency Department Public Use Data File Documentation," 2024. https://ftp.cdc.gov/pub/Health_Statistics/NCHS/Dataset_Documentation/NHAMCS/doc22-ed-508.pdf

[2] CDC/NCHS, "NHAMCS Questionnaires, Datasets, and Documentation." https://www.cdc.gov/nchs/nhamcs/documentation/index.html

[3] FastAPI, "Tutorial - User Guide." https://fastapi.tiangolo.com/tutorial/

[4] React, "Learn React." https://react.dev/learn

[5] PostgreSQL Global Development Group, "PostgreSQL Documentation." https://www.postgresql.org/docs/

[6] Docker, "Docker Compose documentation." https://docs.docker.com/compose/

[7] IETF, "JSON Web Token (JWT), RFC 7519," 2015. https://www.rfc-editor.org/rfc/rfc7519
[8] OWASP Foundation, "API Security Top 10." https://owasp.org/API-Security/
[9] Python Software Foundation, "Python Documentation." https://docs.python.org/3/
[10] SQLAlchemy, "SQLAlchemy 2.0 Documentation." https://docs.sqlalchemy.org/en/20/
[11] Alembic, "Alembic Documentation." https://alembic.sqlalchemy.org/en/latest/
[12] scikit-learn Developers, "scikit-learn User Guide." https://scikit-learn.org/stable/user_guide.html
[13] Vite, "Vite Guide." https://vite.dev/guide/
[14] Uvicorn, "Uvicorn Documentation." https://www.uvicorn.org/
[15] NIST, "Digital Identity Guidelines: Authentication and Lifecycle Management, SP 800-63B." https://pages.nist.gov/800-63-3/sp800-63b.html
[16] IETF, "The Transport Layer Security (TLS) Protocol Version 1.3, RFC 8446," 2018. https://www.rfc-editor.org/rfc/rfc8446
[17] OWASP Foundation, "File Upload Cheat Sheet." https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html
[18] NIST, "Security and Privacy Controls for Information Systems and Organizations, SP 800-53 Rev. 5," 2020. https://csrc.nist.gov/pubs/sp/800/53/r5/upd1/final
[19] WHO, "Ethics and governance of artificial intelligence for health," 2021. https://www.who.int/publications/i/item/9789240029200
[20] CDC/NCHS, "National Hospital Ambulatory Medical Care Survey." https://www.cdc.gov/nchs/ahcd/nhamcs_about.htm

ANNEXURE A - PUBLICATIONS AND INTELLECTUAL PROPERTY

No publication, patent, design registration, or copyright application is claimed as completed in the project handoff. This annexure records the available status and leaves space for later documentary evidence.

Item	Status
Project title	Clinical Urgency and Risk Evaluation (CURE)
Publication status	No publication evidence provided.
Intellectual property status	No patent or design filing evidence provided.
Software repository	https://github.com/Adil-Baig-2197/CURE-app

Evidence or approval reference: _____

ANNEXURE B - STUDENT CONTRIBUTION STATEMENT

Complete this statement with each student's actual contribution before submission. Names and roll numbers are intentionally blank.

Student 1 - Name: _____ Roll No.: _____

Contribution: _____

_____

Student 2 - Name: _____ Roll No.: _____

Contribution: _____

_____

Student 3 - Name: _____ Roll No.: _____

Contribution: _____

_____

Student 4 - Name: _____ Roll No.: _____

Contribution: _____

_____

Guide verification / signature: _____

Contribution Verification

Each student should review the contribution statement above and confirm that it accurately represents the work completed. Names, roll numbers, and signatures are left blank for the team to complete.

Student 1 signature: _____ Date: _____

Student 2 signature: _____ Date: _____

Student 3 signature: _____ Date: _____

Student 4 signature: _____ Date: _____

Guide comments: _____

_____

Guide name and signature: _____

Date: _____

ANNEXURE C - PROJECT OUTPUTS

Outputs include:

- The CURE source repository and Docker Compose setup.
- Backend regression tests and the frontend application.
- Project documentation.
- The research-only ML training/evaluation pipeline.

Synthetic demo files:

- output/pdf/cure_synthetic_demo_lab_report.pdf
- demo-assets/images/cure_synthetic_demo_lab_report.png
- demo-assets/images/cure_synthetic_demo_visit_note.jpg

Run locally from the repository root:

- Configure .env from .env.example.
- Run "docker compose up --build -d".
- Open http://localhost:5173 (API health: http://localhost:8000/api/v1/health).
- Stop with "docker compose down".
- Full Windows PowerShell instructions are in RUN_GUIDE.md.

Research artifacts and CDC data are local/ignored and are not part of application deployment. The experimental model is not loaded by CURE. Use fictional demo records only. CURE is not a diagnosis or emergency service and is not approved for real-patient use.